CMPE 223 / 242
Hands-On Activity 5

1. Given the input array below, trace the Mergesort algorithm for sorting the array in **descending** order. Show the content of the array at the end of each recursive call to the algorithm.

   4 10 9 3 14 5 11 1 8 13 6 7 2 12

2. Given the input array below, trace the Quicksort algorithm for sorting the array in **descending** order. Show the content of the array at the end of each recursive call to the algorithm.

   4 10 9 3 14 5 11 1 8 13 6 7 2 12

3. Give an algorithm to rearrange an array of n keys so that all the **negative keys precede all the nonnegative keys**. Your algorithm must be **in-place**, meaning you cannot allocate another array to temporarily hold the items. Your algorithm must run **in linear time (i.e. *O(N)*)**. You may only use constant size **(i.e. O(1)) additional memory**. First explain how your method works in 2-3 sentences, and then write Java code below. How fast is your algorithm? *Hint: modify the partition function of the quicksort algorithm.*

4. The column on the left contains an input array of 24 strings to be sorted or shuffled; the column on the right contains the strings in sorted order. Each of the other 8 columns contain the contents at some intermediate step during one of the 8 algorithms listed below.

   Match up each algorithm by writing its number under the corresponding column. Use each number at most once.

0	left	hash	flow	byte	byte	byte	byte	byte
1	hash	flip	hash	find	exch	exch	ceil	ceil
2	flip	heap	flip	flip	find	find	edge	edge
3	heap	byte	heap	hash	flip	flip	exch	exch
4	byte	find	byte	heap	flow	hash	find	find
5	find	edge	find	left	hash	heap	flip	flip
6	sort	ceil	edge	sink	heap	left	flow	flow
7	sink	exch	left	sort	left	lifo	hash	hash
8	miss	flow	ceil	exch	left	miss	heap	heap
9	lifo	left	left	flow	lifo	sink	left	left
10	exch	left	exch	left	miss	size	left	left
11	size	left	left	lifo	prim	sort	left	left
12	prim	prim	prim	miss	sink	ceil	lifo	lifo
13	flow	size	size	prim	size	edge	load	load
14	left	trie	lifo	size	sort	flow	loop	loop
15	trie	push	trie	trie	trie	left	miss	miss
16	push	lifo	push	ceil	push	left	push	path
17	ceil	miss	miss	edge	ceil	load	sink	prim
18	left	rank	sink	left	left	loop	trie	push
19	rank	load	rank	load	rank	path	rank	rank
20	load	loop	load	loop	load	prim	sort	sink
21	loop	path	loop	path	loop	push	prim	size
22	path	sink	path	push	path	rank	path	sort
23	edge	sort	sort	rank	edge	trie	size	trie
	----	----	----	----	----	----	----	----
	0							7

(0) Original input	(3) Mergesort (top-down)	(6) Quicksort (Dijkstra 3-way, no shuffle)
(1) Selection sort	(4) Mergesort (bottom-up)	(7) Sorted
(2) Insertion sort	(5) Quicksort (standard, no shuffle)	

5. What is the recurrence relation for the given code below?

```
private static void function_foo(int [] a, int i, int j)
{
   if (i >= j) return;
   int mid = (i + j)/2;
   int k= rand();  // this is a constant time operation that can be considered as 1.
   function_foo (a, i, mid);  // inputs the left half of the original array a[]
   function_foo (a, mid+1, j);  // inputs the right half of the original array a[]
}
```

a) $T_n = 2\,T_n + 2n$;
b) $T_n = T_{n-1} + 1$;
c) $T_n = 2\,T_{n/2} + 1$;
d) $T_n = n \log n + 1$;
e) $T_n = 2\,T_{2n} + n$;

6. Which sorting algorithm is preferred to be used in small and almost sorted arrays? (Hint: due to this property of this sorting algorithm it can increase the performance of other sorting algorithms as well)
   a) Selections sort
   b) Insertion sort
   c) Quicksort
   d) Mergesort
   e) Timsort

7. Suppose we are sorting an array of 10 integers using quicksort, and we have just finished the first partitioning with the array looking like this:

   2, 5, 3, 7, 8, 11, 10, 15, 60, 20

   Which of the following is true?
   a) The pivot could be either the 7 or the 8.
   b) The pivot is 11
   c) The pivot is 10
   d) The pivot is 3
   e) The pivot is 8

8. Suppose that we apply *selection sort* algorithm to the given unsorted array below to sort it in increasing order.

   5, 1, 4, 8, 9, 15, 6, 2

   After the first iteration (phase) of the algorithm, we have the array below:

   1, 5, 4, 8, 9, 15, 6, 2

   Which one is the array after the third iteration (phase) of the algorithm?
   a) 1, 2, 4, 8, 9, 15, 6, 5
   b) 1, 2, 4, 5, 9, 15, 6, 8
   c) 1, 2, 4, 9, 5, 15, 6, 8
   d) 1, 2, 4, 8, 9, 5, 6, 15
   e) 1, 2, 4, 5, 6, 8, 9, 15

9. For the array (5, 1, 4, 8, 9, 15, 6, 2, 46, 78, 11), which value can be selected as the best pivot value for partition process?
   a) 1
   b) 4
   c) 8
   d) 15
   e) 9

10. If we assume that the complexity of a sorting algorithm is $O(nlogn)$ and we run this algorithm on the array sizes (2 million, 4 million, and 8 million). Which one of the execution times can be for this algorithm with these array sizes?
    a) 10, 40, 160 seconds
    b) 10, 80, 640 seconds
    c) 10, 20, 40 seconds
    d) 10, 50, 220 seconds
    e) 10, 160, 1510 seconds

11. If you have an unsorted list of one million unique items, and know that you will only search for an item only three times. Which algorithm would you use?

12. You are given a collection of intervals, where each interval is represented as a pair of integers **[start, end]**. The intervals may overlap with each other. Write an algorithm to merge any overlapping intervals into a single interval.

    Input: The input consists of a collection of intervals, represented as a list of pairs of integers **intervals = [[start1, end1], [start2, end2], ..., [startN, endN]]**, where **start_i** and **end_i** denote the start and end points of the **i**-th interval, respectively.

    Output: Return a new collection of non-overlapping intervals, where each interval is represented similarly as a pair of integers.

    Example:
    Input: intervals = [[1, 3], [2, 6], [8, 10], [15, 18]]
    Output: intervals = [[1, 6], [8, 10], [15, 18]]

    Explanation:
    - The intervals **[1, 3]** and **[2, 6]** overlap, so they are merged into **[1, 6]**.
    - The intervals **[8, 10]** and **[15, 18]** do not overlap with any other intervals, so they remain unchanged.